

Programming in Java (BCAC 0023) Module-1

Faculty Name – Mr. Shivank Chauhan

Syllabus Module 1 : Section 4

Introduction of Classes, Objects and Methods	Defining Class, Adding Variables
	Adding Methods-Default methods, Parameterized methods
	Creating Objects
	Constructors ,Types Of constructor
	Keyword-this ,super
	Accessing Class Members, final Variables and Methods
	Visibility Control
	Static keyword-variable, class, method,

Class

- A class is a user defined blueprint or prototype from which objects are created.
- It represents the set of properties or methods that are common to all objects of one type.
- In terms of object-oriented programming, a class is like a template from which an instance of that class is created at run time Behavior

Syntax to declare class

```
class class-name
{
    properties // or data members or variables
    methods // functions or Behavior
}
```

Here class-name is any valid identifier name.

Methods define the behavior (functions) of the objects.

Properties defines the objects states(values).

```
1 // A sample program to illustrate classes in Java
2 package module2;
3 class Circle
4 {
5     double radius, area;
6     void set_radius(double r) {
7         radius = r;
8     }
9     void display_circle_area() {
10         area = 3.14*radius*radius;
11         System.out.print("\n Radius =" + radius);
12         System.out.println("\n Area =" + area);
13     }
14 }
15 public class Classexample1{
16     public static void main(String args[]){
17         Circle c1 = new Circle();
18         c1.set_radius(2.0);
19         c1.display_circle_area();
20     }
21 }
```

```
1 // A sample program to illustrate classes in Java
2 package module2;
3 class Circle
4 {
5     double radius, area;
6     void set_radius(double r) {
7         radius = r;
8     }
9     void display_circle_area() {
10         area = 3.14*radius*radius;
11         System.out.print("\n Radius =" + radius);
12         System.out.println("\n Area =" + area);
13     }
14 }
15 public class Classexample1{
16     public static void main(String args[]){
17         Circle c1 = new Circle();
18         c1.set_radius(2.0);
19         c1.display_circle_area();
20     }
21 }
```


Adding Variables

- We can add variables by specifying the data type and name of variables

- For example

```
class Circle
```

```
{
```

```
    double radius,area;
```

```
}
```

Here double is the data type and radius, area are variables of the class

Adding Methods

- We can add methods by specifying the following
- 1. Access specifier (public/private/default etc)
- 2. Return type of the methods
- 3. Method name
- 4. Argument list
- 5. Method body
- 6. Other eg static, final etc.

Adding Methods

- The class circle defines two methods set_radius() and display_circle_area

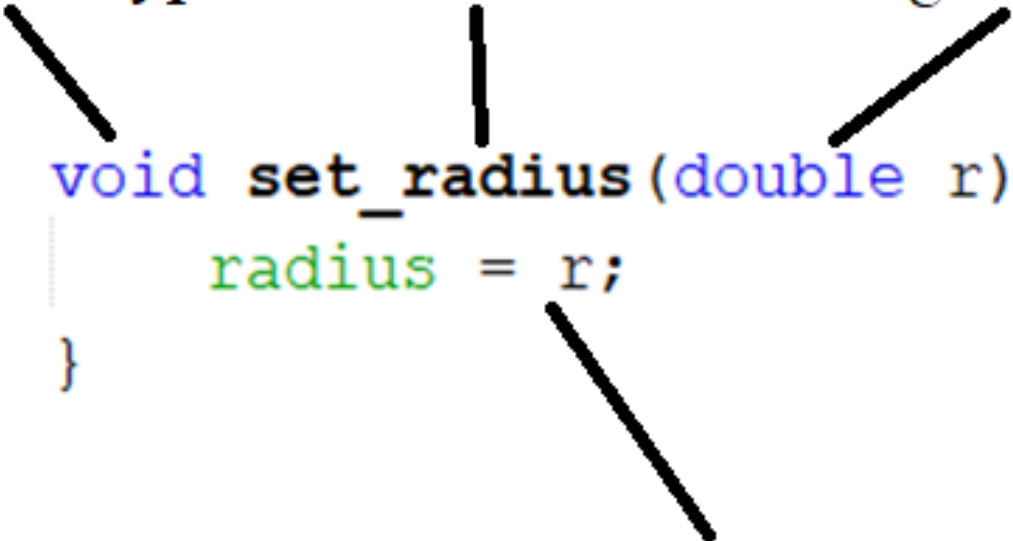
```
void set_radius(double r) {  
    radius = r;  
}  
  
void display_circle_area() {  
    area = 3.14*radius*radius;  
    System.out.print("\n Radius =" + radius);  
    System.out.println("\n Area =" + area);  
}
```

Adding Methods

Return type Method name Argument list

```
void set_radius(double r) {  
    radius = r;  
}
```

Method body



- Here access specifier is not specified so it has default access.

Parameterized methods

- Method definition consists of a method header and a method body.
- The same is shown in the following syntax –
- modifier returnType nameOfMethod (Parameter List) {
- // method body
- }
- Methods with non empty Parameter List are called parameterized methods. Example

```
void set_radius(double r) {  
    radius = r;  
}
```

Creating Objects

- In Java, an object is created from a class.
- We can create objects by using class-name, object-name, = operator, new keyword and a constructor of the class
- For example : The first line is creating an object of type Circle.

```
Circle c1 = new Circle();  
c1.set_radius(2.0);  
c1.display_circle_area();
```

Constructor

- Constructors are used to initialize the object's state/data members.
- A constructor also contains collection of statements that are executed at time of Object creation.
- So constructors are used to assign values to the class variables at the time of object creation, either explicitly done by the programmer or by Java itself (default constructor).

Constructor Example

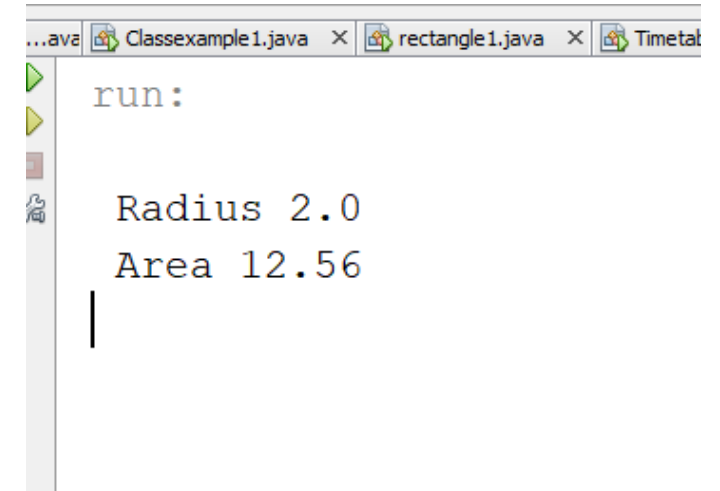
```
class circle
{
    double radius,area;
    circle()
    {
        radius=0;
    }
}
```

```
class circle
{
    double radius,area;
    circle()
    {
        radius=0;
    }
    circle(double r)
    {
        radius=r;
    }
    void calarea()
    {
        area = 3.14 *radius*radius;
        System.out.print("\n Radius " + radius);
        System.out.print("\n Area " + area);
    }
}
```



```
class circle
{
    double radius,area;
    circle()
    {
        radius=0;
    }
    circle(double r)
    {
        radius=r;
    }
    void calarea(){
        area = 3.14 *radius*radius;
        System.out.print("\n Radius " + radius);
        System.out.print("\n Area " + area);
    }
}

public class constructor1
{
    public static void main(String args[]){
        circle c1 = new circle(2);
        c1.calarea();
    }
}
```



```
run:
Radius 2.0
Area 12.56
```

When is a Constructor called ?

- A constructor is invoked at the time of object or instance creation.
- Each time an object is created using `new()` keyword at least one constructor is invoked to assign initial values to the data members of the same class.

Rules for writing Constructor:

- Constructor(s) of a class must have same name as the class name in which it resides.
- A constructor in Java can not be abstract, final, static and Synchronized.
- Access modifiers can be used in constructor declaration to control its access i.e. which other class can call the constructor.

How constructors are different from methods in Java?

- Constructor(s) must have the same name as the class within which it is defined while it is not necessary for the method in java.
- Constructor(s) do not return any type while method(s) have the return type or **void** if it does not return any value.
- Constructor is called only once at the time of Object creation while method(s) can be called any number of times.

Types of constructor

- **No-argument constructor:**
- **Parameterized Constructor**
- **Constructor Overloading**

No-argument constructor: default constructor

- A constructor that has no parameter is known as default constructor. If we don't define a constructor in a class, then compiler creates **default constructor(with no arguments)** for the class.
- If we write a constructor with arguments or no-arguments then the compiler does not create a default constructor.
- Example

No-argument constructor: default constructor

```
class circle
{
    double radius,area;
    circle()
    {
        radius=0;
    }
}
```

Parameterized Constructor:

- A constructor that has parameters is known as parameterized constructor.
- If we want to initialize fields of the class with your own values, then use a parameterized constructor.

Parameterized Constructor:

```
class circle
{
    double radius,area;
    circle(double r)
    {
        radius=r;
    }
}
```

Exercise : class Rectangle

- Create a class Rectangle and add following
- Data Members
 1. Length, Breadth and Area
- Methods
 1. Calculate area
 2. Display
- Constructor
 1. Default
 2. Parameterized

Exercise : class BankAccount

- Create a class BankAccount and add following
- Data Members
 1. Name as string
 2. Balance as double
- Methods
 1. Deposit
 2. Withdraw
 3. Display
- Constructor
 1. Default
 2. Parameterised

Some Examples

- Create class for
- Rectangle
- Bank Account

The this Keyword

- There can be a lot of usage of **java this keyword**. In java, this is a **reference variable** that refers to the current object.

The this Keyword

- In the body of a method definition, sometimes you need to refer to the object that contains the method (in other words, the object itself).
- To refer to the object in its own method, use the this keyword where you normally would refer to an object's name.
- The this keyword refers to the current object which is calling that method.
- Because this is a reference to the current instance of a class, only use it inside the body of an instance method definition.

Usage of java this keyword

- Here is given the 6 usage of java this keyword.
 1. this can be used to refer current class instance variable.
 2. this can be used to invoke current class method (implicitly)
 3. this() can be used to invoke current class constructor.
 4. this can be passed as an argument in the method call.
 5. this can be passed as argument in the constructor call.
 6. this can be used to return the current class instance from the method.

```
public class MyClass {  
    int x;  
    // Constructor with a parameter  
    public MyClass(int x)  
    {  
        this.x = x;  
    }  
  
    // Call the constructor  
    public static void main(String[] args) {  
        MyClass myObj = new MyClass(5);  
        System.out.println("Value of x = " + myObj.x);  
    }  
}
```


- Here is given the 6 usage of java this keyword.
1. this can be used to refer current class instance variable. (Constructor)
 2. this can be used to invoke current class method (implicitly) (Method call)
 3. this() can be used to invoke current class constructor.
 4. this can be passed as an argument in the method call. (pass in method)
 5. this can be passed as argument in the constructor call.
 6. this can be used to return the current class instance from the method.

this can be used to refer current class instance variable.

```
class Student{  
    int rollno;  
    String name;  
    float fee;  
    Student(int rollno,String name,float fee){  
        this.rollno=rollno;  
        this.name=name;  
        this.fee=fee;  
    }  
    void display(){System.out.println(rollno+" "+name+" "+fee);}  
}
```

this can be passed as argument in the construct call.

```
class A{  
    A(){System.out.println("hello a");}  
    A(int x){  
        this();  
        System.out.println(x);  
    }  
}
```

Static Keyword

- The static keyword in Java is used for memory management mainly.
- We can apply static keyword with variables or methods .
- The static keyword belongs to the whole class but not to an instance of the class.

Static variables

- The static variable can be used to refer to the common property of all objects (which is not unique for each object), for example, the company name of employees, college name of students, etc.
- The static variable gets memory only once in the class area at the time of class loading.
- It makes your program **memory efficient** (i.e., it saves memory).

Understanding the problem without static varia

```
class Student{  
    int rollno;  
    String name;  
    String college="GLA";  
}
```

- Suppose there are 500 students in my college, now all instance data members will get memory each time when the object is created.
- All students have its unique rollno and name, so instance data member is good in such case.
- Here, "college" refers to the common property of all objects. If we make it static, this field will get the memory only once.

//Java Program to demonstrate the use of static variable

```
//Java Program to demonstrate the use of static variable
```

```
class Student{
    int rollno;//instance variable
    String name;
    static String college ="GLA";//static variable
    //constructor
    Student(int r, String n){
        rollno = r;
        name = n;
    }
    //method to display the values
    void display (){
        System.out.println(rollno+" "+name+" "+college);
    }
}
```

```
}
}
//Test class to show the values of objects
public class TestStaticVariable1{
    public static void main(String args[]){
        Student s1 = new Student(111,"Karan");
        Student s2 = new Student(222,"Aryan");
        //we can change the college of all objects by the single line of code
        //Student.college="Galgotia";
        s1.display();
        s2.display();
    }
}
```

//Java Program to demonstrate the use of static var

```
class Student{
    int rollno;//instance variable
    String name;
    static String college ="GLA";//static variable
    //constructor
    Student(int r, String n){
        rollno = r;
        name = n;
    }
    //method to display the values
    void display (){
        System.out.println(rollno+" "+name+" "+college) ;
    }
}
```



```
//Java Program to demonstrate the use of static variable
class Student{
    int rollno;//instance variable
    String name;
    static String college ="GLA";//static variable
    //constructor
    Student(int r, String n){
        rollno = r;
        name = n;
    }
    //method to display the values
    void display (){
        System.out.println(rollno+" "+name+" "+college);
    }
}
//Test class to show the values of objects
public class TestStaticVariable1{
    public static void main(String args[]){
        Student s1 = new Student(111,"Karan");
        Student s2 = new Student(222,"Aryan");
        //we can change the college of all objects by the single line of code
        //Student.college="Galgotia";
        s1.display();
        s2.display();
    }
}
```

Static
Variable

```
run:
111 Karan GLA
222 Aryan GLA
BUILD SUCCESSFUL (
```

Java static method

- If you apply static keyword with any method, it is known as static method.
- A static method belongs to the class rather than the object of a class.
- A static method can be invoked without the need for creating an instance of a class.
- A static method can access static data member and can change the value of it.

Java Program to demonstrate the use of a static method.

```

package static1;

//Java Program to demonstrate the use
of a static method.

class Student{
    int rollno;
    String name;
    static String college = "GLA";

    //static method to change the value of
    static variable
    static void change(String college2){
        college = college2;
    }

    //constructor to initialize the variable
    Student(int r, String n){
        rollno = r;
        name = n;
    }

    //method to display values
    void
    display(){System.out.println(rollno+"
    "+name+" "+college);}
}

//Test class to create and display the
values of object
public class TestStaticMethod{
    public static void main(String args[])
    {
        //creating objects
        Student s1 = new
        Student(111,"Karan");
        Student s2 = new
        Student(222,"Aryan");
        Student.change("GLA
        University");//calling change method

        Student s3 = new
        Student(333,"Sonoo");

        //calling display method
        s1.display();
        s2.display();
        s3.display();
    }
}

```

```
//Java Program to demonstrate the use of a static method.
class Student{
    int rollno;
    String name;
    static String college = "GLA";
    //static method to change the value of static variable
    static void change(String college2){
        college = college2;
    }
    //constructor to initialize the variable
    Student(int r, String n){
        rollno = r;
        name = n;
    }
    //method to display values
    void display(){System.out.println(rollno+" "+name+" "+college);}
}
```

```
//Test class to create and display the values of object
public class TestStaticMethod{
    public static void main(String args[])
    {
        //creating objects
        Student s1 = new Student(111,"Karan");
        Student s2 = new Student(222,"Aryan");
        Student.change("GLA University");//calling change method
        Student s3 = new Student(333,"Sonoo");
        //calling display method
        s1.display();
        s2.display();
        s3.display();
    }
}
```

run:

111 Karan GLA University

222 Aryan GLA University

333 Sonoo GLA University

BUILD SUCCESSFUL (total time: 0 seconds)

Another example

```
class Calculate
{
    static int cube(int x)
    {
        return x*x*x;
    }
    static int square(int x)
    {
        return x*x;
    }
    public static void main(String args[])
    {
        int result=Calculate.cube(5);
        System.out.println(result);
        result=Calculate.square(9);
        System.out.println(result);
    }
}
```

run:

125

81

BUILD SUCCESSFUL (tot

Another example

```
class X
{
    int x1;
    static int y1=1200;
    static void change_y1(int a)
    {
        y1=a;
    }
    void change_x1(int b)
    {
        x1=b;
    }
    void display()
    {
        System.out.print("\n x1 = " +
            x1);
        System.out.print("\n y1 = " +
            y1);
    }
}

public class staticVariables
{
    public static void main(String
        args[])
    {
        X obj1 = new X();
        obj1.change_x1(100);
        System.out.print("\n Object 1");
        obj1.display();
    }
}
```

```
X obj2 = new X();
obj2.change_x1(200);
X.change_y1(1500);
System.out.print("\n Object 2");
obj2.display();

System.out.print("\n Object 1");
obj1.display();
```

Restrictions for the static method

There are two main restrictions for the static method.

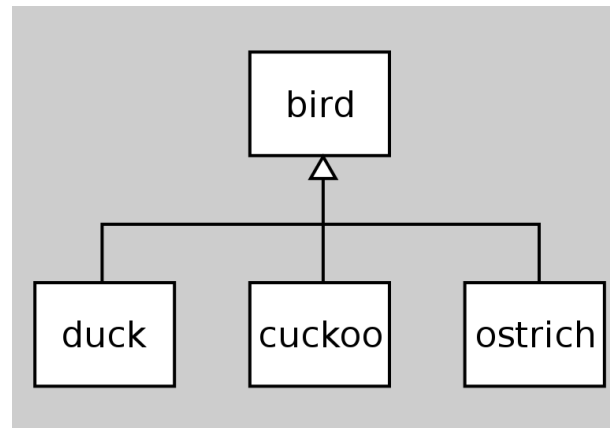
They are:

1. The static method can not use non static data member or call non-static method directly.
2. this and super cannot be used in static context.

Inheritance

Inheritance

- Inheritance in Java is a mechanism in which one object acquires all the properties and behaviors of a parent object.
- The idea behind inheritance in Java is that you can create new classes that are built upon existing classes.
- When you inherit from an existing class, you can reuse methods and fields of the parent class.



Inheritance



Inheritance

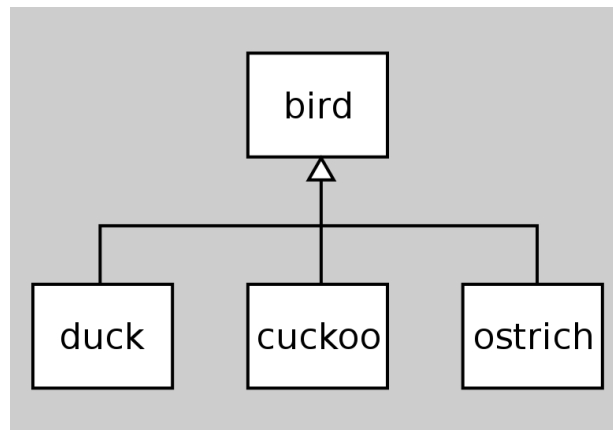


Inheritance



Inheritance

- Moreover, you can add new methods and fields in your current class also.
- It is an important part of OOPs (Object Oriented programming system).
- Inheritance represents the IS-A relationship which is also known as a parent-child relationship.

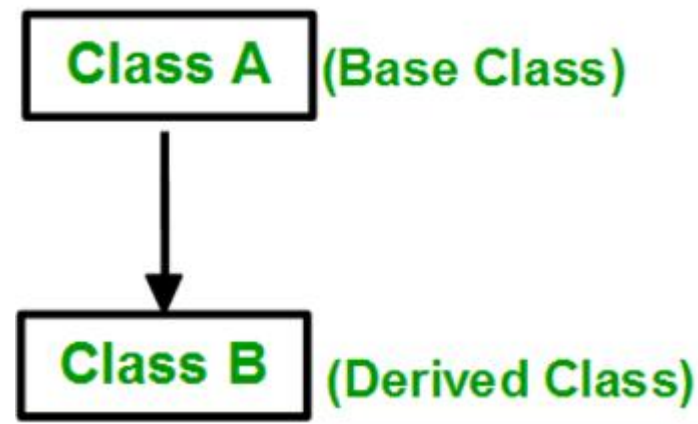


Inheritance

- Why use inheritance in java
 1. For Method Overriding (so runtime polymorphism can be achieved).
 2. For Code Reusability.

Terms used in Inheritance

- **Class:** A class is a group of objects which have common properties. It is a template or blueprint from which objects are created.
- **Sub Class/Child Class:** Subclass is a class which inherits the other class. It is also called a derived class, extended class, or child class.
- **Super Class/Parent Class:** Superclass is the class from where a subclass inherits the features. It is also called a base class or a parent class.



Terms used in Inheritance

- **Reusability:** As the name specifies, reusability is a mechanism which facilitates you to reuse the fields and methods of the existing class when you create a new class.
- You can use the same fields and methods already defined in the previous class.

The syntax of Java Inheritance

```
class Subclass-name extends Superclass-name
{
    //methods and fields
}
```

```
class Animal
{
    void eat()
    {
        System.out.println("eating...");
    }
}
```

```
class Dog extends Animal
{
    void bark()
    {
        System.out.println("barking...");
    }
}
```

```
class Animal
{
    void eat() {
        System.out.println("eating...");
    }
}
class Dog extends Animal
{
    void bark()
    {
        System.out.println("barking...");
    }
}
public class Inheritance1 {
    public static void main(String args[])
    {
        Dog d=new Dog();
        d.bark();
        d.eat();
    }
}
```

```
run:
barking...
eating...
BUILD SUCCESSFUL (total
```

Examples

Shape

Length, Breadth

Rectangle

Area

Cal_Area()

Display()

Set_data()

Examples

Shape

Side

Square

Area

Cal_Area()

Display()

Set_data()

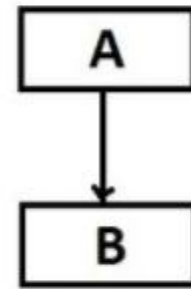
The syntax of Java Inheritance

```
class Subclass-name extends Superclass-name
{
    //methods and fields
}
```

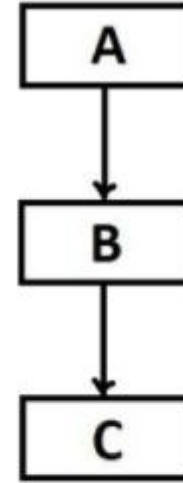
- The **extends keyword** indicates that you are making a new class that derives from an existing class.
- The meaning of "extends" is to increase the functionality.
- In the terminology of Java, a class which is inherited is called a parent or superclass, and the new class is called child or subclass.

Types of Inheritance in Java

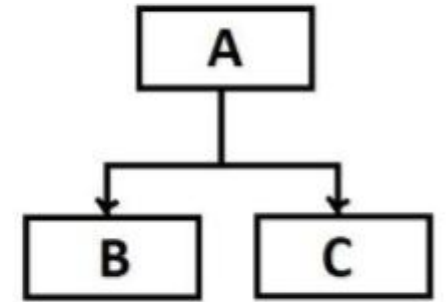
1. Single Inheritance
2. Multilevel Inheritance
3. Hierarchical Inheritance
4. Multiple Inheritance(interface)
5. Hybrid Inheritance (interface)



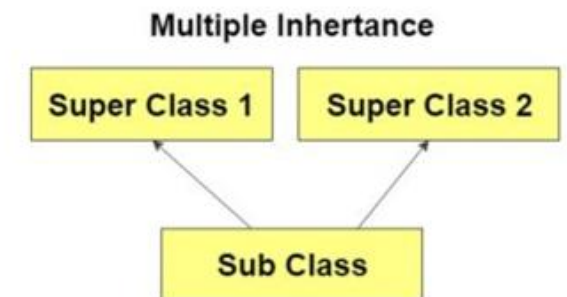
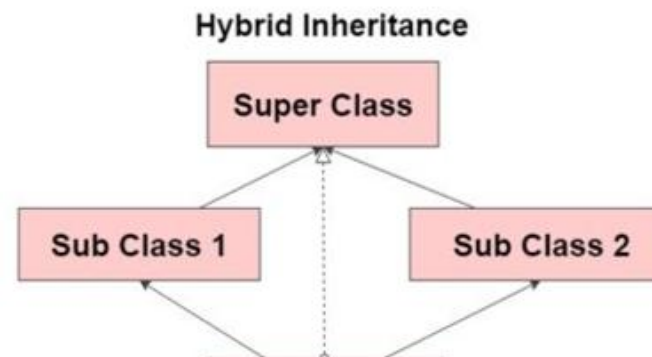
simple
inheritance



multi-level inheritance



hierarchical inheritance

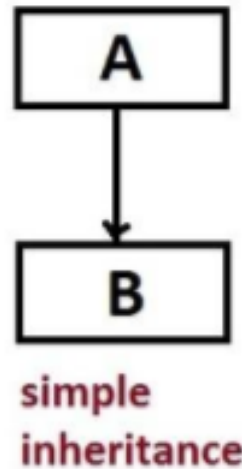


Single Inheritance

- When a class inherits another class, it is known as a *single inheritance*.
- In the example Dog class inherits the Animal class, so there is the single inheritance

```
class Animal
{
    void eat()
    {
        System.out.println("eating...");
    }
}
```

```
class Dog extends Animal
{
    void bark()
    {
        System.out.println("barking...");
    }
}
```



Examples

Shape

Length, Breadth

Rectangle

Area

Cal_Area()

Display()

Set_data()

Examples

Shape

Side

Square

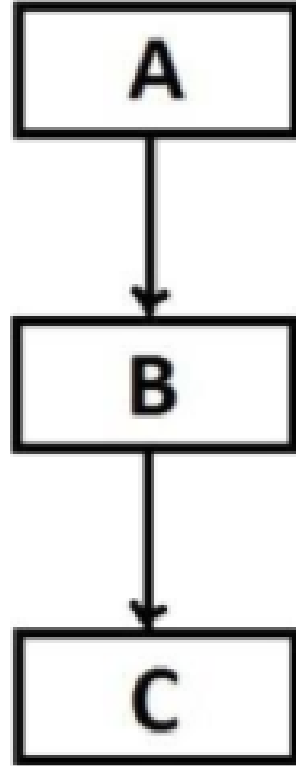
Area

Cal_Area()

Display()

Set_data()

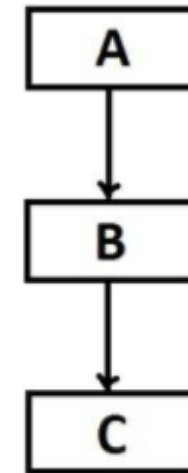
Multilevel Inheritance



multi-level inheritance

Multilevel Inheritance

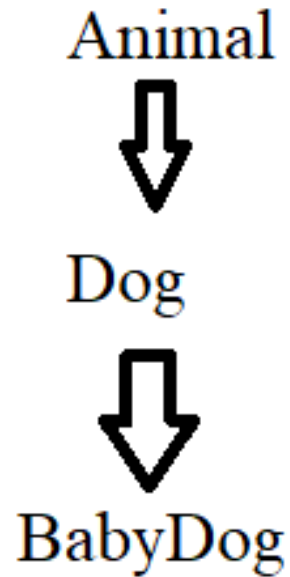
- When there is a chain of inheritance, it is known as *multilevel inheritance*.
- Or
- When a derived class is again derived then it is called multilevel inheritance
- Here class C inherits class B and class B inherits class A so it is called multilevel inheritance.



multi-level inheritance

Multilevel Inheritance

- As you can see in the example given below, BabyDog class inherits the Dog class which again inherits the Animal class, so there is a multilevel inheritance.



Multilevel Inheritance

```

package Inheritance;

class Animal
{
    void eat()
    {
        System.out.println("eating...");
    }
}

class Dog extends Animal
{
    void bark()
    {
        System.out.println("barking...");
    }
}

class BabyDog extends Dog
{
    void weep()
    {
        System.out.println("weeping...");
    }
}

public class MultipleInheritance
{
    public static void main(String
args[])
    {
        BabyDog d=new BabyDog();
        d.weep();
        d.bark();
        d.eat();
    }
}

```

Multilevel Inheritance

Line

Data Members

Length

Member Functions

Line()

Line(int L)

set_length(int L)

Rectangle extends Line

Data Members

Breadth

Area

Member Functions

set_length_and_breadth(int L, Int B)

cal_area()

display()

Box extends Rectangle

Data Members

Height

volume

Member Functions

Box()

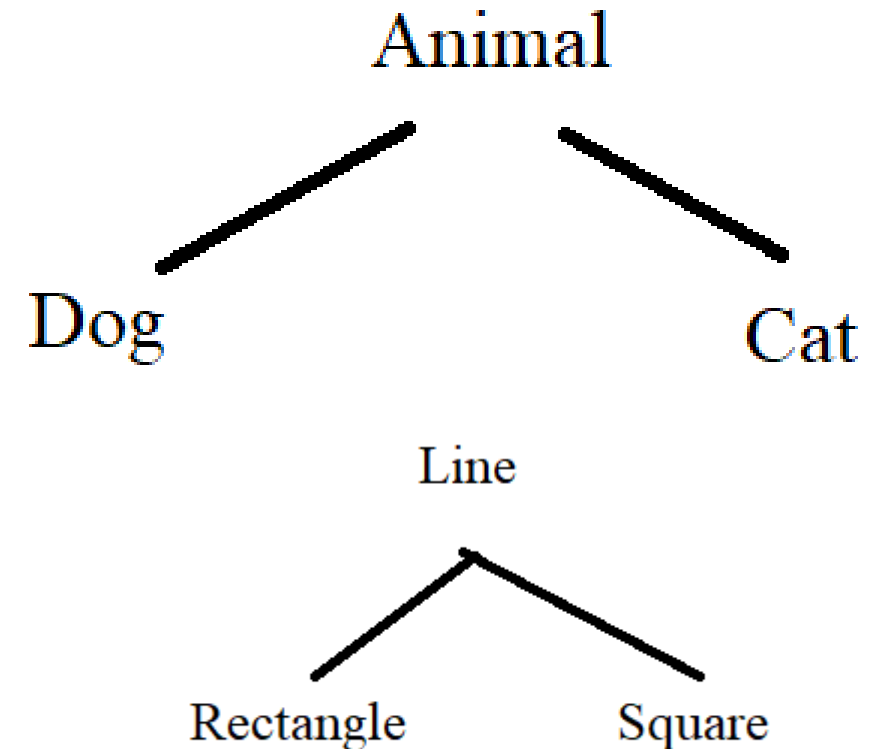
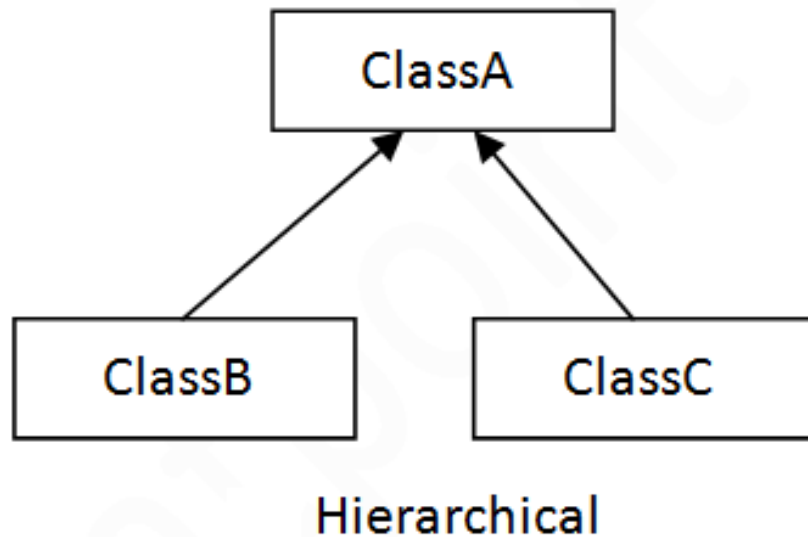
Box(int L,int B,int H)

cal_volume()

display()

Hierarchical Inheritance

- When two or more classes inherits a single class, it is known as *hierarchical inheritance*.
- For example



Hierarchical Inheritance

```
class Animal{  
    void eat(){System.out.println("eating...");}  
}  
class Dog extends Animal{  
    void bark(){System.out.println("barking...");}  
}  
class Cat extends Animal{  
    void meow(){System.out.println("meowing...");}  
}  
class TestInheritance3{  
    public static void main(String args[]){  
        Cat c=new Cat();  
        c.meow();  
        c.eat();  
    }  
}
```

```
run:  
meowing...  
eating...  
BUILD SUCCESSFUL (total ti
```

Remaining Topics Sequence

- Super()
- final Variables and Methods
- Visibility Control
- Abstract Class
- Dynamic method Dispatching
- Method Over Loading & Overriding
- Interfaces
- Packages

Super Keyword in Java

- The super keyword in Java is a reference variable which is used to refer immediate parent class object.
- Whenever you create the instance of subclass, an instance of parent class is created implicitly which is referred by super reference variable.

Usage of Java super Keyword

1. `super` can be used to refer immediate parent class instance variable.
2. `super` can be used to invoke immediate parent class method.
3. `super()` can be used to invoke immediate parent class constructor.

Usage of Java super Keyword

1. super can be used to refer immediate parent class instance variable

```
package super1;
class Animal
{
    String color="white";
}
class Dog extends Animal
{
    String color="black";
    void printColor() {
        System.out.println(color); //prints color of Dog class
        System.out.println(super.color); //prints color of Animal class
    }
}
class TestSuper1{
    public static void main(String args[]) {
        Dog d=new Dog();
        d.printColor();
    }
}
```

```
package super1;
class Animal
{
    String color="white";
}
class Dog extends Animal
{
    String color="black";
    void printColor(){
        System.out.println(color);//prints
```

color of Dog class

```
        System.out.println(super.color);//pr
        ints color of Animal class
    }
}
class TestSuper1{
    public static void main(String
    args[]){
        Dog d=new Dog();
        d.printColor();
    }
}
```


Usage of Java super Keyword

1. super can be used to invoke immediate parent class method.

```
package super1;
class Animal
{
void eat() {
    System.out.println("eating...");
}
class Dog extends Animal{
    void eat() {
        System.out.println("eating bread...");
    }
void bark() {System.out.println("barking...");}
void work()
{
    super.eat();
    bark();
}
}
class TestSuper2{
public static void main(String args[]){
    Dog d=new Dog();
    d.work();
}}
```

run:

eating...

barking...

BUILD SUCCESSFUL (

super can be used to invoke immediate parent class method.

```
package super1;

class Animal
{
    void eat(){
        System.out.println("eating...");
    }
}

class Dog extends Animal{
    void eat() {
        System.out.println("eating bread...");
    }
}

void bark(){System.out.println("barking...");}
void work()
```

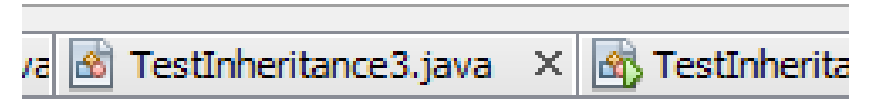
```
{
    super.eat();
    bark();
}

class TestSuper2{
    public static void main(String args[]){
        Dog d=new Dog();
        d.work();
    }
}
```

Usage of Java super Keyword

`super()` can be used to invoke immediate parent class constructor.

```
package super1;
class Animal
{
    Animal()
    {
        System.out.println("animal is created");
    }
class Dog extends Animal
{
    Dog()
    {
        super();
        System.out.println("dog is created");
    }
}
class TestSuper3{
public static void main(String args[]){
Dog d=new Dog();
}}
```



TestInheritance3.java × TestInheritance

```
run:
animal is created
dog is created
BUILD SUCCESSFUL (tot
```

super is used to invoke parent class constructor.

```
class Animal{
    Animal(){System.out.println("animal is created");}
}
class Dog extends Animal{
    Dog(){
        super();
        System.out.println("dog is created");
    }
}
class TestSuper3{
    public static void main(String args[]){
        Dog d=new Dog();
    }
}
```

Example using Super

```
package p2;

class Line
{
    int length;

    Line()
    {
        System.out.println("\n Default constructor of Line");
        length=0;
    }

    Line(int l)
    {
        length=l;
        System.out.println("\n Parameterised constructor of Line");
    }

    void display()
    {
        System.out.println("\n Length of Line = " + length);
    }
}

class Rectangle extends Line
{
```

```
    int breadth,area;

    Rectangle()
    {
        super();
        System.out.println("\n Default constructor of Rectangle");
        breadth=0;
    }

    Rectangle(int l,int b)
    {
        super(l);
        System.out.println("\n Parameterised constructor of Rectangle");
        breadth=b;
    }

    void cal_area()
    {
        area=length*breadth;
        System.out.println("\n Area = " + area);
    }

    void display()
    {
        System.out.println("\n Length of Rectangle = " + length);
```

```
        System.out.println("\n Breadth of Rectangle = " + breadth);
        System.out.println("\n Area of Rectangle = " + area);
    }
}

class Box extends Rectangle
{
    int height,volume;

    Box()
    {
        //super();
        super.length=0;
        super.breadth=0;
        super.cal_area();
        System.out.println("\n Default constructor of Box");
        height=0;
    }

    Box(int l,int b,int h)
    {
        //super(l,b);
        super.length = l;
        super.breadth = b;
```

```
        System.out.println("\n Parameterised constructor of Box");
        height=h;
        super.cal_area();
        cal_volume();
    }

    void cal_volume()
    {
        volume=length*breadth*height;
        System.out.println("\n Volume = " + volume);
    }
}

public class super1
{
    public static void main(String args[])
    {
        Box b = new Box(2,3,4);
    }
}
```

Method Overloading and Overriding

Method Overloading in Java

- If a class has multiple methods having same name but different in parameters, it is known as Method Overloading.
- Reference
- <https://www.javatpoint.com/method-overloading-in-java>

```
class Cat{  
  
    public void Sound(){  
        System.out.println("meow");  
    }  
  
    //overloading method  
    public void Sound(int num){  
        for(int i=0; i<2;i++){  
            System.out.println("meow");  
        }  
    }  
}
```

OVERLOADING

Same method
name but
different
parameters

Method Overloading in Java

- Suppose you have to perform addition of numbers
- But there can be any number of arguments
- if you write the method such as
- `a(int,int)` for two parameters
- `b(int,int,int)` for three parameters
- then it may be difficult for you as well as other programmers to understand the behavior of the method because its name differs.

Method Overloading in Java

- So define two methods using method overloading
 - add(int,int)
 - add(int,int,int)
-
- It increases the readability / understandability of the program.

Method Overloading in Java

- Advantage of method overloading
- Method overloading increases the readability of the program.

Method Overloading in Java

- There are two ways to overload the method in java

1. By changing number of arguments
2. By changing the data type

Eg.

Add(int a,int b)

Add(double a, double b)

Add(int a,int b,int c)

Method Overloading in Java

1. By changing number of arguments

we have created two methods, first add() method performs addition of two numbers and second add() method performs addition of three numbers.

```
int add(int a,int b){return a+b;}
```

```
int add(int a,int b,int c){return a+b+c;}
```

```
class Adder
```

```
{  
    int add(int a,int b){return a+b;}  
    int add(int a,int b,int c){return a+b+c;}  
}
```

```
class TestOverloading1
```

```
{  
    public static void main(String[] args)  
    {  
        Adder adder_object = new Adder();  
        System.out.println(adder_object.add(11,11));  
        System.out.println(adder_object.add(11,11,11));  
    }  
}
```

run:

22

33

BUILD SUCCESSFUL

Method Overloading in Java

1. By changing number of arguments

```
class Adder
{
    int add(int a,int b){return a+b;}
    int add(int a,int b,int c){return a+b+c;}
}
class TestOverloading1
{
    public static void main(String[] args)
    {
        Adder adder_object = new Adder();
        System.out.println(adder_object.add(11,11));
        System.out.println(adder_object.add(11,11,11));
    }
}
```

Method Overloading in Java

2. By changing the data type

In this example, we have created two methods that differs in data type. The first add method receives two integer arguments and second add method receives two double arguments.

```
int add(int a, int b){return a+b;}
```

```
double add(double a, double b){return a+b;}
```

Method Overloading in Java

2. By changing the data type

```
class Adder{  
    int add(int a, int b){return a+b;}  
    double add(double a, double b){return a+b;}  
}  
class TestOverloading1  
{  
    public static void main(String[] args)  
    {  
        Adder adder_object = new Adder();  
        System.out.println(adder_object.add(11,11));  
        System.out.println(adder_object.add(2.5,3.1));  
    }  
}
```

run:

22

5.6

BUILD SUCCESS

Method Overloading in Java : Example

```
package Overloading;

class FD
{
    public static double cal_interest(double p, double t)
    {
        double r=6.0;
        double si=0;
        si =(p*r*t)/100;
        return si;
    }
    public static double cal_interest(double p, double t, double r)
    {
        double si=0;
        si =(p*r*t)/100;

        return si;
    }
}

public class Overload1
{
    public static void main(String args[])
    {
        System.out.print("\n " + FD.cal_interest(10000,5));
        System.out.print("\n " + FD.cal_interest(10000,5,7));
    }
}
```

Method Overloading in Java

2. By changing the data type

```
class Adder{
    int add(int a, int b){return a+b;}
    double add(double a, double b){return a+b;}
}
class TestOverloading1
{
    public static void main(String[] args)
    {
        Adder adder_object = new Adder();
        System.out.println(adder_object.add(11,11));
        System.out.println(adder_object.add(2.5,3.1));
    }
}
```

Method Overloading in Java

- Q) Why Method Overloading is not possible by changing the return type of method only?
- In java, method overloading is not possible by changing the return type of the method only because of ambiguity.
- Let's see how ambiguity may occur:

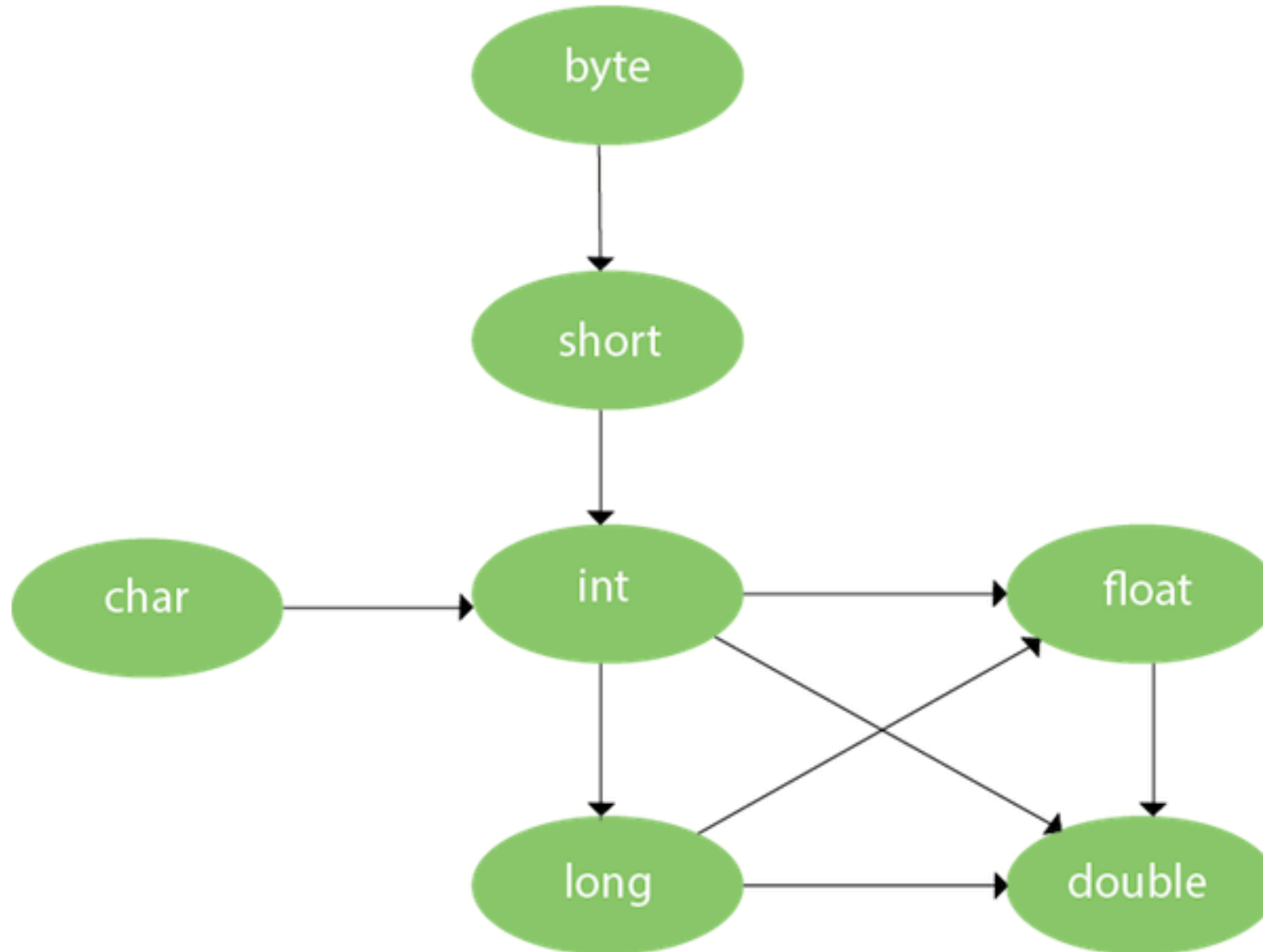
```
1  class Adder{
2      int add(int a,int b){return a+b;}
3      double add(int a,int b){return a+b;}
4  }
5  class TestOverloading1
6  {
7      public static void main(String[] args)
8      {
9          Adder adder_object = new Adder();
10         System.out.println(adder_object.add(11,11));
11         // Ambiguity occur
12     }
13 }
```

Why Method Overloading is not possible by changing the return type of method only?

```
class Adder{  
    int add(int a,int b){return a+b;}  
    double add(int a,int b){return a+b;} // error  
}  
  
class TestOverloading1{  
    public static void main(String[] args)  {  
        Adder adder_object = new Adder();  
        System.out.println(adder_object.add(11,11));  
        // Ambiguity occur  
    }  
}
```


Method Overloading and Type Promotion

- One type is promoted to another implicitly if no matching datatype is found.

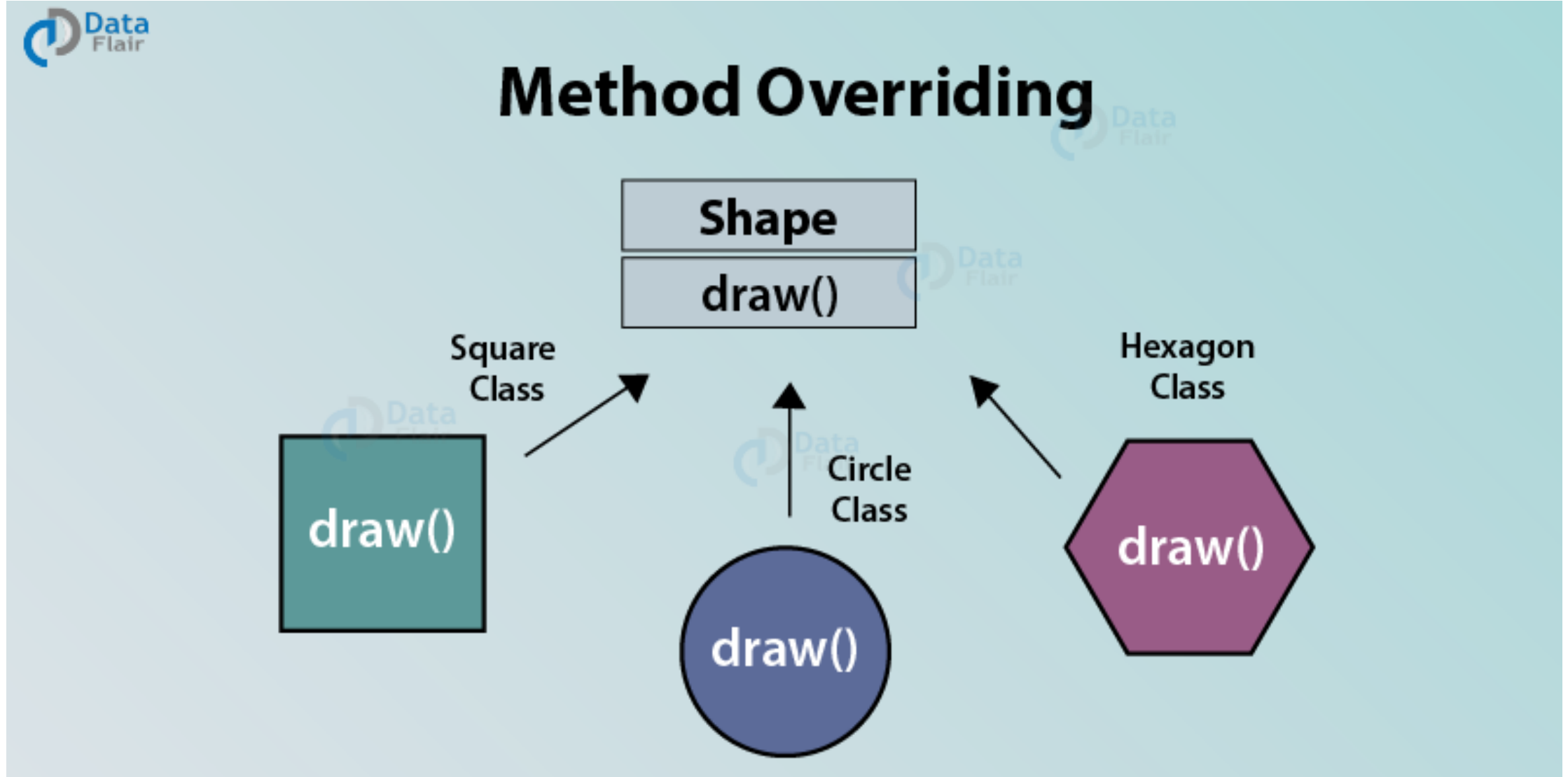


Example of Method Overloading with Type Promotion in case of ambiguity

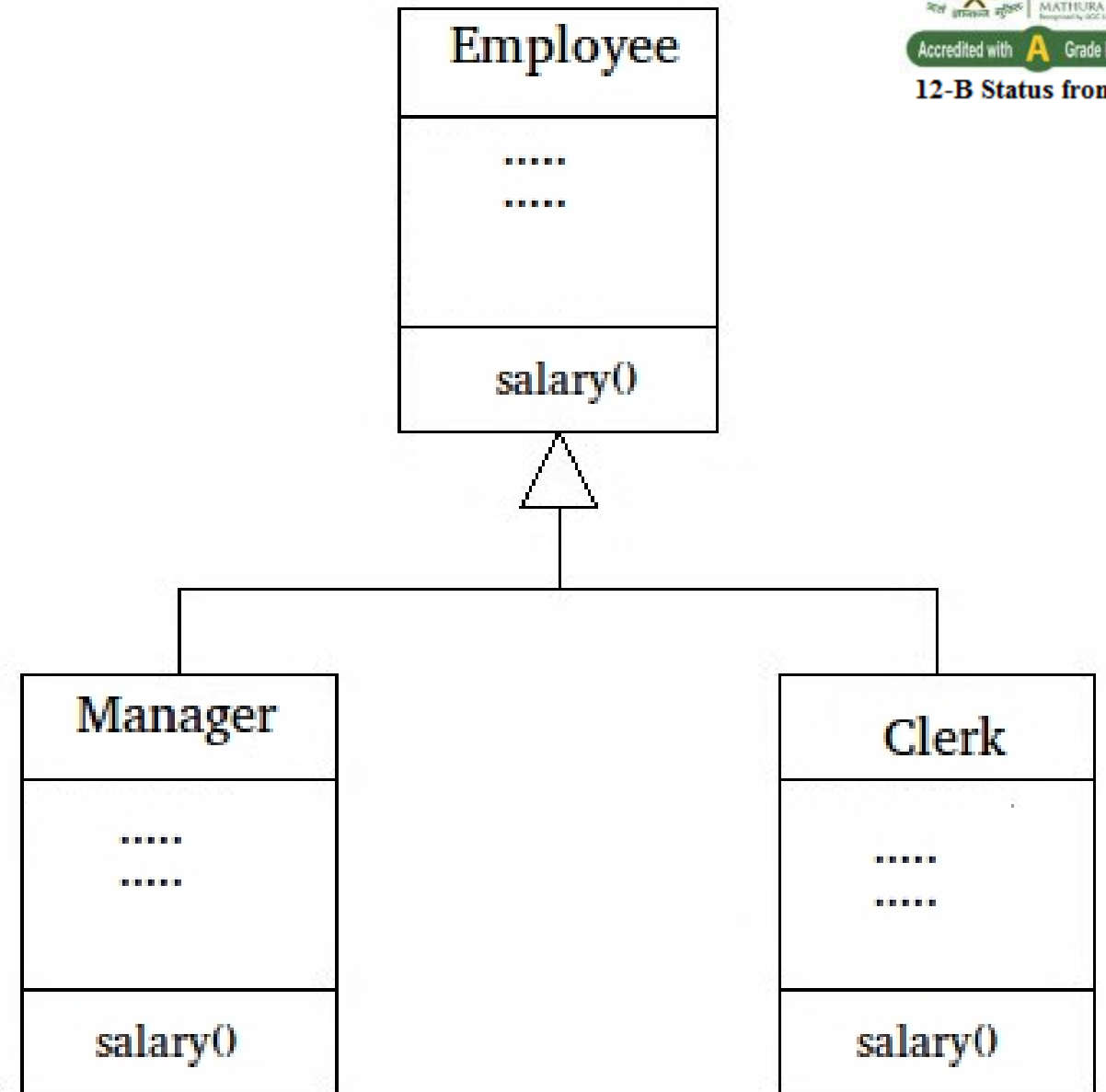
```
class OverloadingCalculation3{  
    void sum(int a,long b){System.out.println("a method invoked");}  
    void sum(long a,int b){System.out.println("b method invoked");}  
  
    public static void main(String args[]){  
        OverloadingCalculation3 obj=new OverloadingCalculation3();  
        obj.sum(20,20);//now ambiguity  
    }  
}
```

Output:Compile Time Error

Method Overriding



Method Overriding



Method Overriding

- If subclass (child class) has the same method as declared in the parent class, it is known as method overriding in Java.
- In other words, If a subclass provides the specific implementation of the method that has been declared by one of its parent class, it is known as method overriding.

Usage of Java Method Overriding

1. Method overriding is used to provide the specific implementation of a method which is already provided by its superclass.
2. Method overriding is used for runtime polymorphism

Rules for Java Method Overriding

- The method must have the same name as in the parent class
 - The method must have the same parameter as in the parent class.
 - There must be an IS-A relationship (inheritance).
-
- Reference
 - <https://www.javatpoint.com/method-overriding-in-java>

Example of method overriding - 1

```
class Vehicle {  
    void run() {  
        System.out.println("Vehicle is running");  
    }  
}  
class Bike2 extends Vehicle {  
    void run() {  
        System.out.println("Bike is running safely");  
    }  
}  
public class MethodOverriding1 {  
    public static void main(String args[]) {  
        Bike2 obj = new Bike2();//creating object  
        obj.run();//calling method  
    }  
}
```

```
run:  
Bike is running safely  
BUILD SUCCESSFUL (total
```


Example of method overriding

//Java Program to illustrate the use of
Java Method Overriding

```
class Vehicle {  
    void run(){  
        System.out.println("Vehicle is  
running");  
    }  
}  
  
class Bike2 extends Vehicle {  
    void run(){  
        System.out.println("Bike is running
```

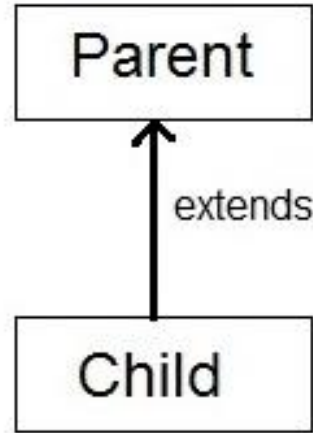
```
safely");  
    }  
}  
  
public class MethodOverriding1{  
    public static void main(String args[]) {  
        Bike2 obj = new Bike2();//creating  
object  
        obj.run();//calling method  
    }  
}
```

Dynamic method Dispatching : Using Method Overriding

- Dynamic method dispatch is a mechanism by which a call to an overridden method is resolved at runtime.
- This is how java implements runtime polymorphism.
- When an overridden method is called by a reference, java determines which version of that method to execute based on the type of object it refer to.

Upcasting in Java

- When **Parent** class reference variable refers to **Child** class object, it is known as **Upcasting**. It is used to achieve run time polymorphism.



```
Parent p = new Parent( );
```

```
Child c = new Child( );
```

```
Parent p = new Child( );
```

←
Upcasting

```
Child c = new Parent( );
```

incompatible type

```
class Animal{
    public void move() {
        System.out.println("Animals can move");
    }
}

class Dog extends Animal {
    public void move() {
        System.out.println("Dogs can walk and run");
    }
}

public class MethodOverriding2 {
    public static void main(String args[]){
        Animal a = new Animal();
        Animal b = new Dog();
        a.move(); // runs the method in Animal class
        b.move(); // runs the method in Dog class
    }
}
```

Upcasting

Run time polymorphism / Dynamic
Dispatching

run:

Animals can move

Dogs can walk and run

BUILD SUCCESSFUL (total ti

Dynamic method Dispatching : Using Method Overriding

```
package overriding;

class Animal
{
    public void move()
    {
        System.out.println("Animals can move");
    }
}

class Dog extends Animal
{
    public void move()
    {
        System.out.println("Dogs can walk and run");
    }
}
```

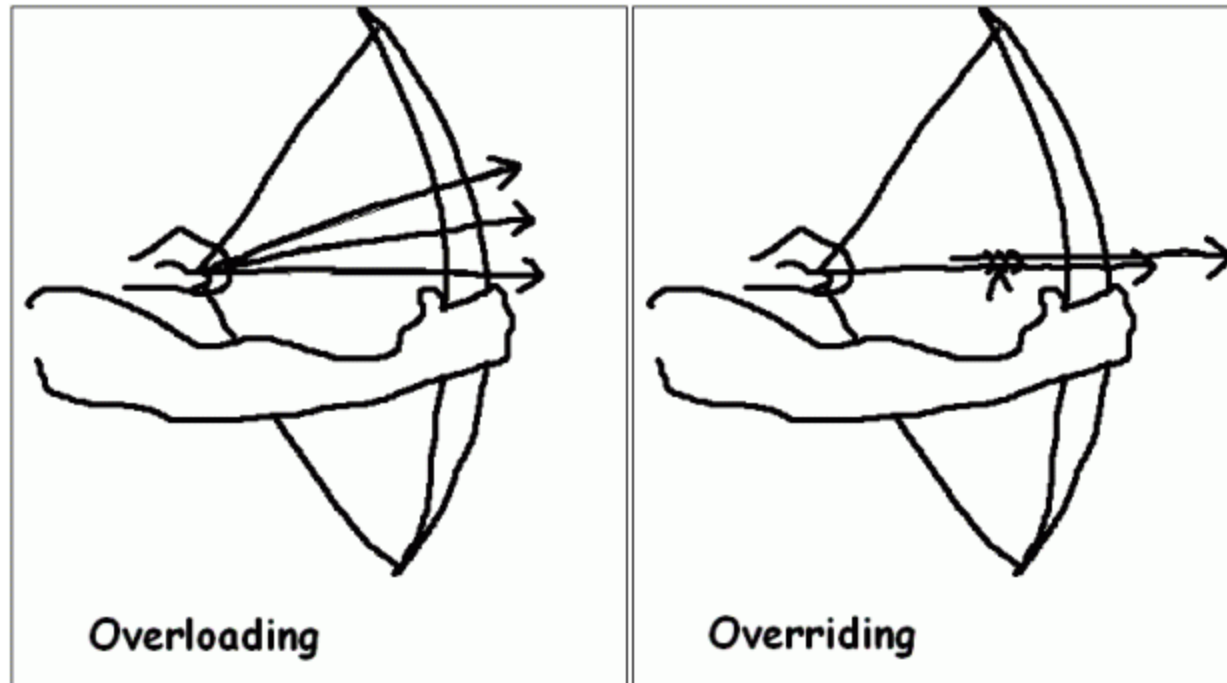
```
}

public class MethodOverriding2
{
    public static void main(String args[])
    {
        Animal a = new Animal(); // Animal reference and
        object
        Animal b = new Dog(); // Animal reference but Dog
        object
        a.move(); // runs the method in Animal class
        b.move(); // runs the method in Dog class
    }
}
```

Static binding and Dynamic binding in Java!

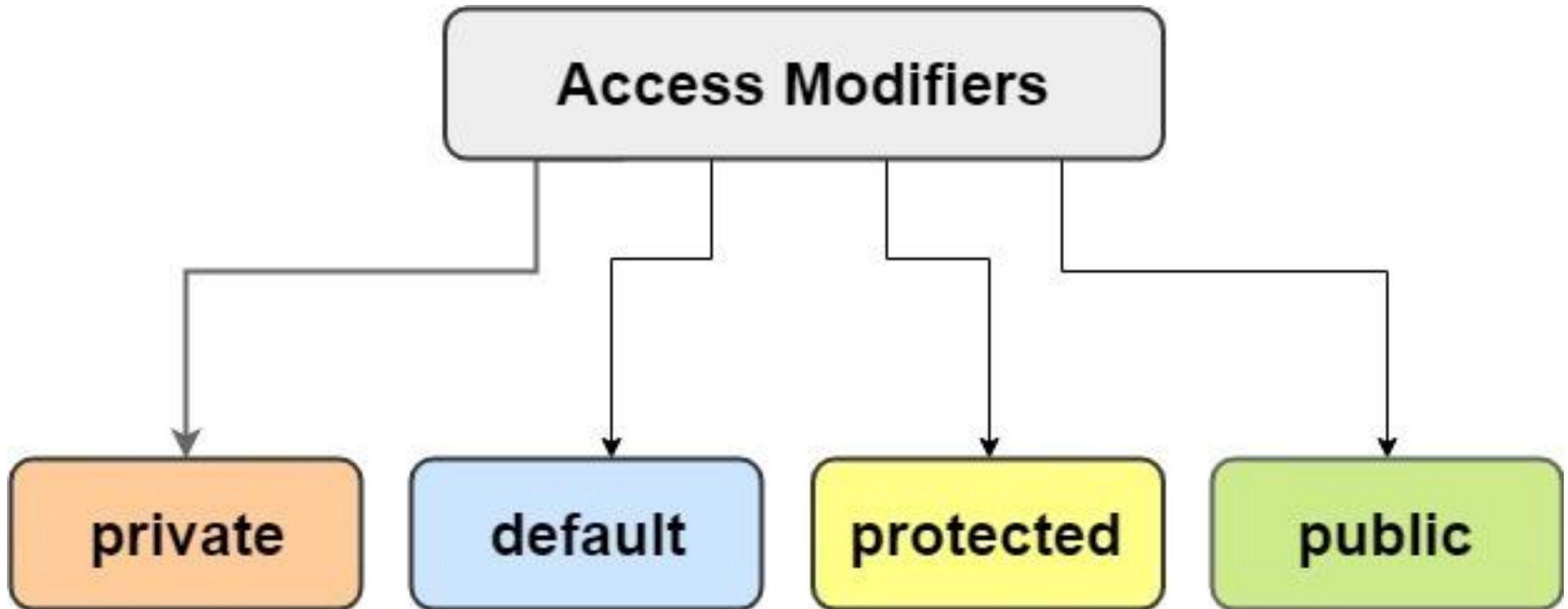
- Static binding in Java occurs during compile time while dynamic binding occurs during runtime.
- Static binding uses type(Class) information for binding while dynamic binding uses instance of class(Object) to resolve calling of method at run-time.
- Overloaded methods are bonded using static binding while overridden methods are bonded using dynamic binding at runtime.

Overloading Vs Overriding



No.	Method Overloading	Method Overriding
1)	Method overloading is used to increase the readability of the program.	Method overriding is used to provide the specific implementation of the method that is already provided by its super class.
2)	Method overloading is performed within class.	Method overriding occurs in two classes that have IS-A (inheritance) relationship.
3)	In case of method overloading, parameter must be different.	In case of method overriding, parameter must be same.
4)	Method overloading is the example of compile time polymorphism.	Method overriding is the example of run time polymorphism.

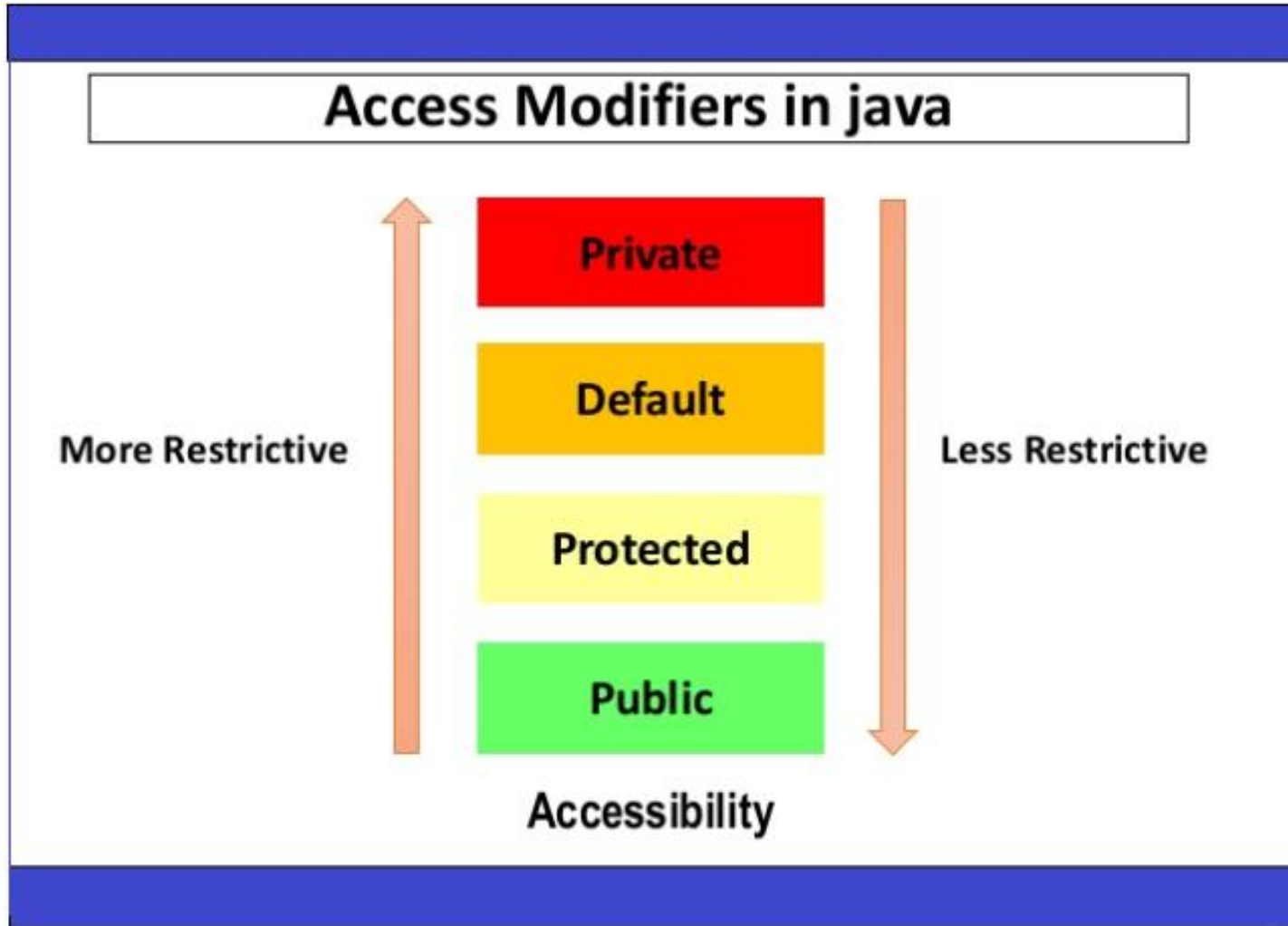
Visibility Control / Access Modifiers in Java



Visibility Control / Access Modifiers in Java

- The access modifiers in Java specifies the accessibility or scope of a field, method, constructor, or class.
- We can change the access level of fields, constructors, methods, and class by applying the access modifier on it.
- There are four types of Java access modifiers:
 1. Private
 2. Default
 3. Protected
 4. Public

Visibility Control / Access Modifiers in Java



Visibility Control / Access Modifiers in Java

- **Private:**
 - The access level of a private modifier is only within the class.
 - It cannot be accessed from outside the class.
- **Default:**
 - The access level of a default modifier is only within the package.
 - It cannot be accessed from outside the package.
 - If you do not specify any access level, it will be the default.

Visibility Control / Access Modifiers in Java

- **Protected:**
 - The access level of a protected modifier is within the package and outside the package through child class.
 - If you do not make the child class, it cannot be accessed from outside the package.
- **Public:**
 - The access level of a public modifier is everywhere.
 - It can be accessed from within the class, outside the class, within the package and outside the package.

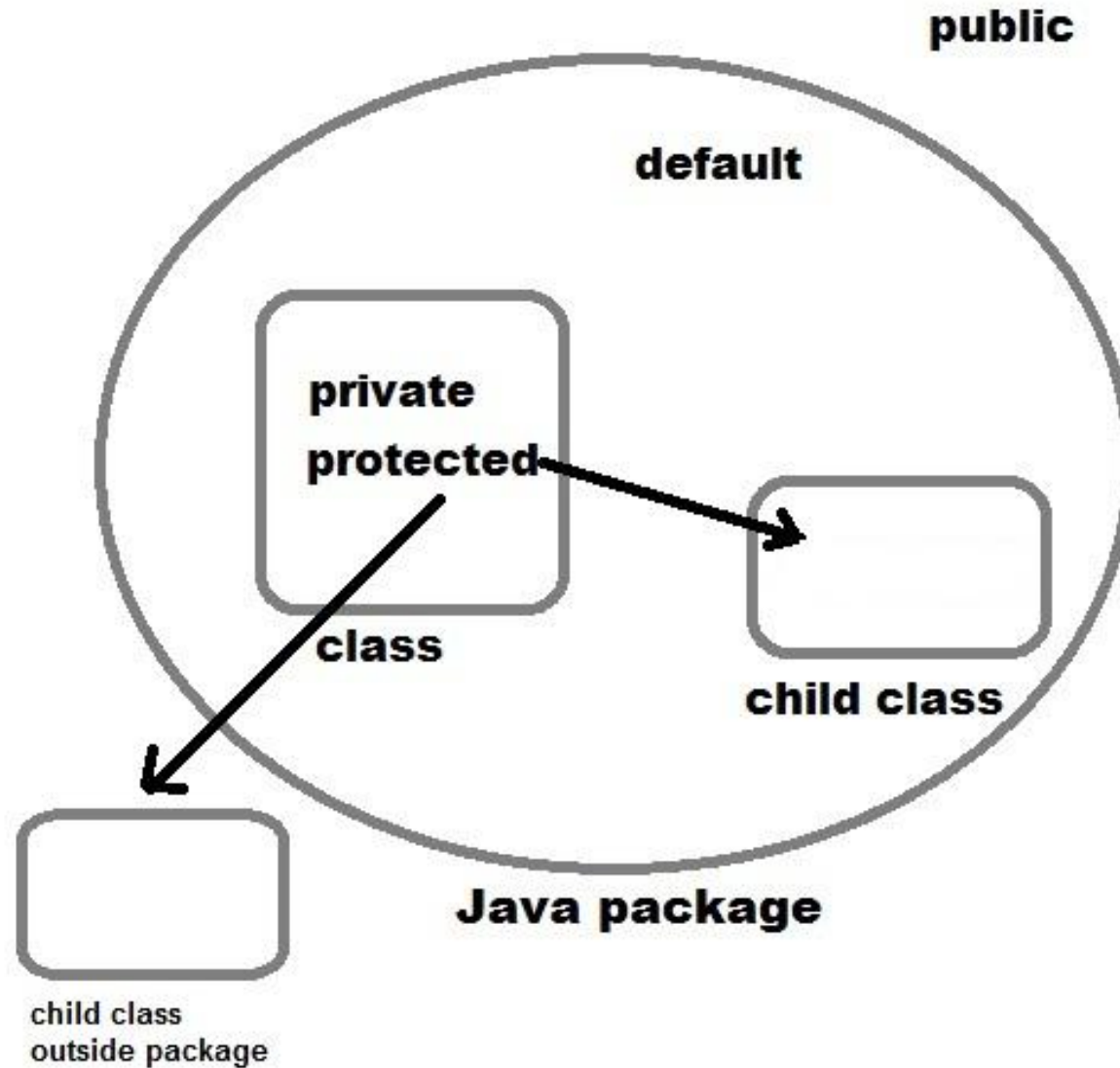
Visibility Control / Access Modifiers in Java

- **Private:** The access level of a private modifier is only within the class. It cannot be accessed from outside the class.
- **Default:** The access level of a default modifier is only within the package. It cannot be accessed from outside the package. If you do not specify any access level, it will be the default.
- **Protected:** The access level of a protected modifier is within the package and outside the package through child class. If you do not make the child class, it cannot be accessed from outside the package.
- **Public:** The access level of a public modifier is everywhere. It can be accessed from within the class, outside the class, within the package and outside the package.

Visibility Control / Access Modifiers in Java

Access Modifier	within class	within package	outside package by subclass only	outside package
Private	Y	N	N	N
Default	Y	Y	N	N
Protected	Y	Y	Y	N
Public	Y	Y	Y	Y

Visibility Control / Access Modifiers in Java



Final Keyword

Final Keyword In Java

- The **final keyword** in java is used to restrict the user.
- Final can be:
 1. Variable.
 2. Method
 3. class

1. final variable

- If you make any variable as final, you cannot change the value of final variable(It will be constant).

Example

```
class A
{
    final int a=90;//final variable
    void run()
    {
        a=400;
    }
    public static void main(String x[])
    {
        A a1=new A();
        a1.run();
    }
}
```

Output: Compile Time Error

2. final method

- If you make any method as final, you cannot override it.

Example

```
class A
{
    final void run()
    {
        System.out.println("running");
    }
}
```

Cont....

```
class B extends A
{
    void run()
    {
        System.out.println("running safely with 100kmph");
    }
    public static void main(String a[])
    {
        B b1= new B();
        b1.run();
    }
}
```

Output: Compile Time Error

3. final class

- If you make any class as final, you cannot extend it.

Example

```
final class A
{
    void run()
    {
        System.out.println("running");
    }
}
```

Cont....

```
class B extends A
{
    void run()
    {
        System.out.println("running safely with 100kmph");
    }
    public static void main(String a[])
    {
        B b1= new B();
        b1.run();
    }
}
```

Output: Compile Time Error

Abstraction in Java

Abstraction in Java

- Abstraction is a process of hiding the implementation details and showing only functionality to the user.
- For example, sending SMS where you type the text and send the message. You don't know the internal processing about the message delivery.

Ways to achieve Abstraction

- There are two ways to achieve abstraction in java.
 1. Abstract class (0 to 100%)
 2. Interface (100%)

1. Abstract class in Java

- A class which is declared as keyword abstract is known as an **abstract class**.
- It can have abstract and non-abstract methods.

Points to Remember

- An abstract class must be declared with an abstract keyword.
- It can have abstract and non-abstract methods.
- We can not create the object of abstract class.

Cont...

- It needs to be extended in another class and its abstract method should be implemented in that class.

Example of abstract class

abstract class A

{

}

Abstract Method in Java

- A method which is declared as abstract and does not have body is known as an abstract method.

- **Example of abstract method**

abstract void A();//no method body and abstract

Example

```
abstract class A
{
    abstract void run();
    void display()
    {
        System.out.println("Bike");
    }
}
class B extends A
{
    void run()
    {
        System.out.println("running safely");
    }
    public static void main(String args[])
    {
        B b1 = new B();
        b1.run();
        b1. display();
    }
}
```

Cont...

Output:

running safely

Bike

2. Interface in Java

- It has only abstract methods, not method with body.
- It is used to achieve 100 % abstraction and multiple inheritance in Java.

Why use Java interface?

- It is used to achieve 100% abstraction.
- By interface, we can support the functionality of multiple inheritance.

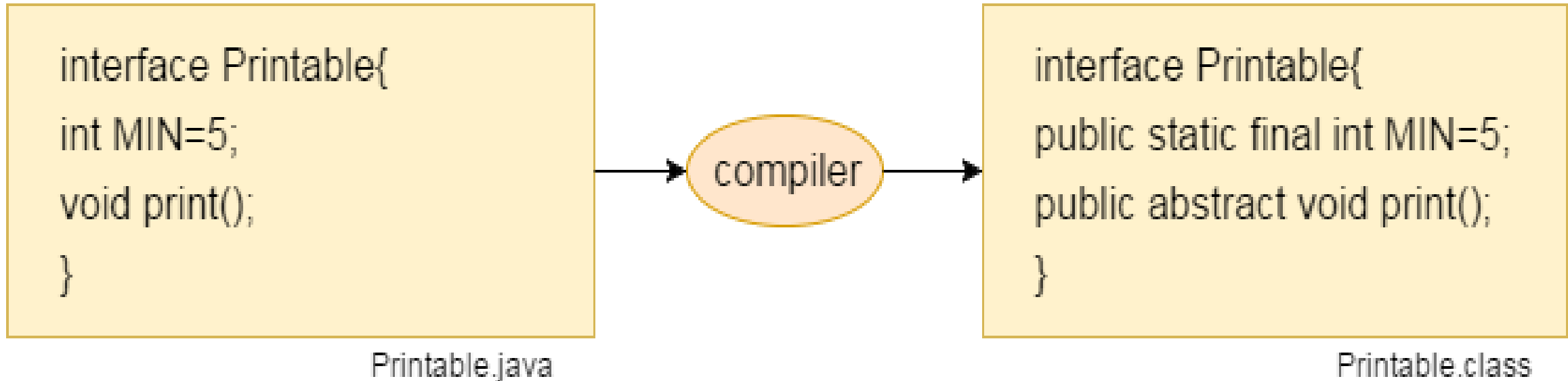
Note:

- Like **abstract classes**, object of interfaces **can not** be created.
- It needs to be implements in a class and its method should be implemented in that class.
- Interface methods are by default abstract and public.

Cont...

- Interface attributes are by default public, static and final.

Example:



Example

```
interface A
{
    void print();
}
class B implements A
{
    public void print()
    {
        System.out.println("Hello");
    }
    public static void main(String a[])
    {
        B b1 = new B();
        b1.print();
    }
}
```

Cont...

Output:

Hello

Multiple inheritance with the help of Interface

```
interface X
{
    void display();
}
interface Y
{
    void display();
}
class C implements X, Y
{
    public void display()
    {
        System.out.println("Hello");
    }
    public static void main(String x[])
    {
        C c1 = new C();
        c1.display();
    }
}
```

Output:
Hello